데이터베이스(SQL)

5장. 내장 함수 기초

학습 목표

1) 문자열 함수를 사용할 수 있다.
2) 숫자 함수를 사용할 수 있다.
3) 날짜 함수를 사용할 수 있다.
4) CASE WHEN으로 조건에 따라 다른 값을 반환할 수 있다.
5) COALESCE로 NULL을 대체값으로 처리할 수 있다.
6) NULLIF로 특정 값을 NULL로 변환할 수 있다.
7) CAST로 데이터 타입을 변환할 수 있다.
8) TO_CHAR로 날짜와 숫자의 표시 형식을 바꿀 수 있다.
9) 문자열 연결 ||과 :: 형변환을 사용할 수 있다.

공식 문서: PostgreSQL 내장 함수

전체 흐름

1) 문자열 함수 → 2) 형변환 → 3) 숫자 함수 → 4) 날짜 함수

5) CASE → 6) COALESCE · NULLIF → 7) TO_CHAR · TO_DATE

01 문자열 함수

문자열 함수 전체 목록 (1/2)

함수	역할	예시	결과
UPPER	대문자 변환	UPPER('abc')	ABC
LOWER	소문자 변환	LOWER('ABC')	abc
INITCAP	단어 첫 글자만 대문자	INITCAP('hello world')	Hello World
LENGTH	문자열 길이	LENGTH('M14860')	6
TRIM	양쪽 공백 제거	TRIM(' A ')	A
LTRIM	왼쪽 공백 제거	LTRIM(' A ')	A (오른쪽 공백은 남는다)
RTRIM	오른쪽 공백 제거	RTRIM(' A ')	A (왼쪽 공백은 남는다)

공식 문서: 문자열 함수

문자열 함수 전체 목록 (2/2)

함수	역할	예시	결과
SUBSTRING	부분 문자열 추출	SUBSTRING('M14860',2,3)	148
LEFT	왼쪽부터 N글자	LEFT('M14860',3)	M14
RIGHT	오른쪽부터 N글자	RIGHT('M14860',3)	860
REPLACE	문자열 치환	REPLACE('M14860','M','설비-')	설비-14860
CONCAT	문자열 연결	CONCAT('A','B')	AB
CONCAT_WS	구분자로 연결	CONCAT_WS('-','A','B')	A-B

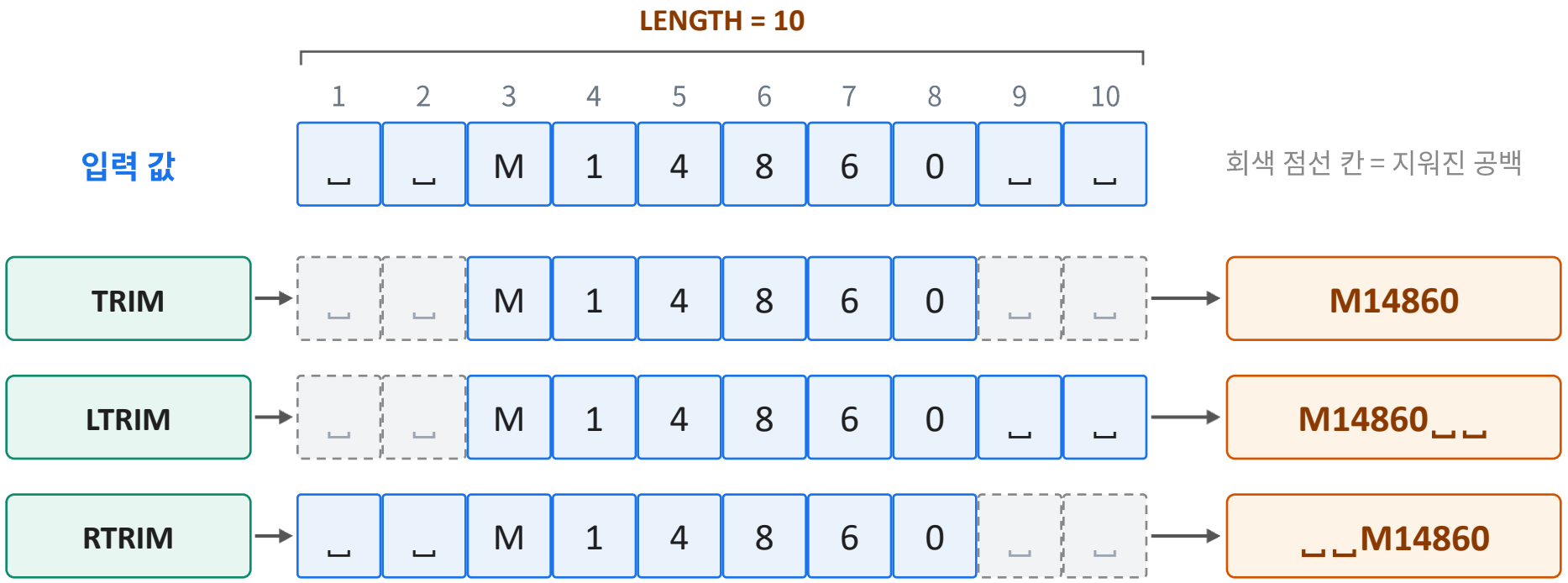
대소문자 변환

```
-- 대소문자 변환
SELECT
UPPER('hello world') AS 대문자, -- HELLO WORLD
LOWER('HELLO WORLD') AS 소문자, -- hello world
INITCAP('hello world') AS 첫글자대문자; -- Hello World
```

공백 처리

```
-- 공백 처리
SELECT
LENGTH('  M14860  ') AS 공백포함, -- 10
TRIM('  M14860  ') AS 양쪽제거, -- M14860
LTRIM('  M14860  ') AS 왼쪽만, -- M14860␣␣
RTRIM('  M14860  ') AS 오른쪽만; -- ␣␣M14860
```

TRIM · LTRIM · RTRIM의 공백 제거

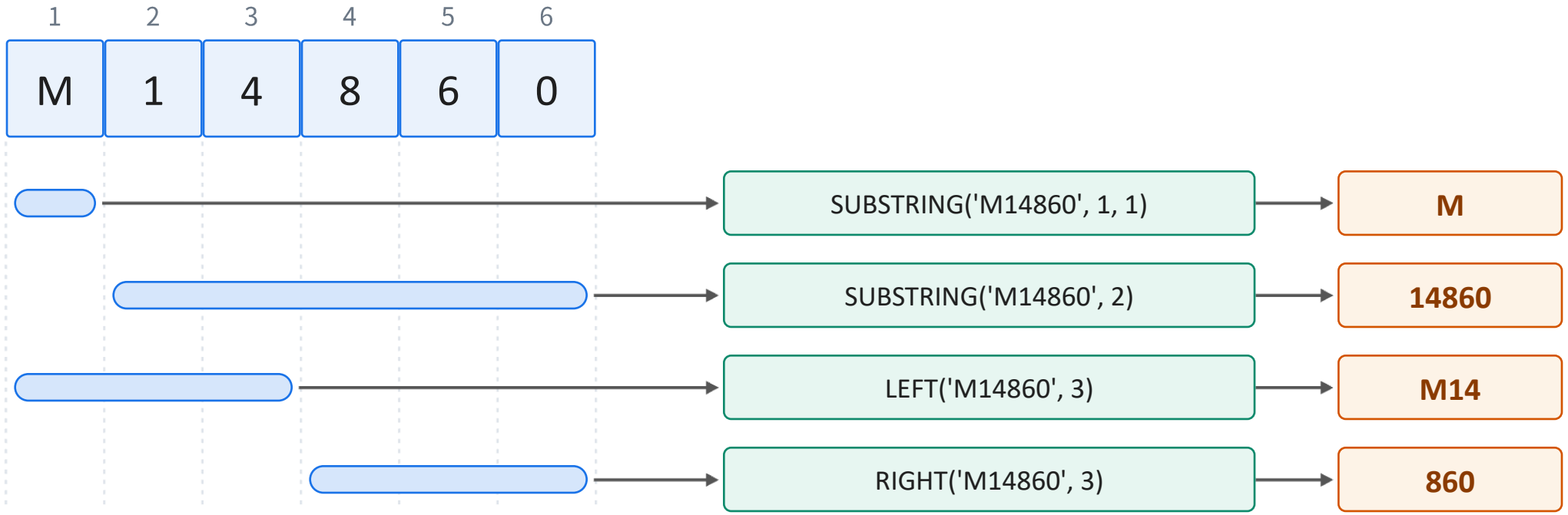


L은 왼쪽, R은 오른쪽, TRIM은 양쪽

문자열 자르기

```
-- 문자열 자르기
SELECT
SUBSTRING('M14860', 1, 1) AS 첫글자, -- M
SUBSTRING('M14860', 2) AS 두번째부터, -- 14860
LEFT('M14860', 3) AS 왼쪽3글자, -- M14
RIGHT('M14860', 3) AS 오른쪽3글자; -- 860
```

문자 위치와 자르기



SQL의 문자 위치는 1부터 센다

문자열 연결

```
-- 문자열 연결
SELECT
CONCAT('설비-', 'M14860') AS CONCAT사용, -- 설비-M14860
'설비-' || 'M14860' AS 파이프연산자, -- 설비-M14860
CONCAT_WS('-', '설비', 'M14860', 'H') AS 구분자포함; -- 설비-M14860-H
```

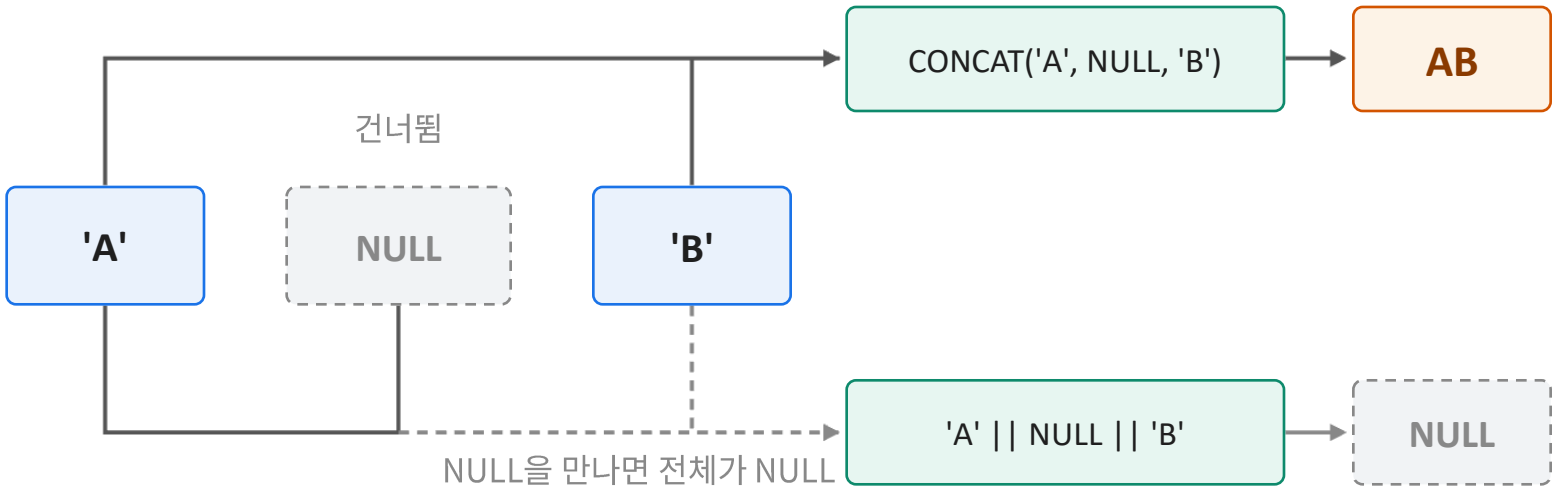
CONCAT과 ||의 NULL 처리 차이

방식	NULL 처리	결과
CONCAT('A', NULL, 'B')	NULL 무시	AB
'A' || NULL || 'B'	전체가 NULL	NULL

```
SELECT
CONCAT('A', NULL, 'B') AS CONCAT결과, -- AB
'A' || NULL || 'B' AS 파이프결과, -- NULL
'A' || COALESCE(NULL, '') || 'B' AS 안전한연결; -- AB
```

|| 를 쓸 때 컬럼에 NULL이 있으면 전체가 NULL => COALESCE로 감싸서 처리.

CONCAT과 ||의 NULL 처리



COALESCE로 감싼 안전한 연결

'A' || COALESCE(NULL, '') || 'B'

→ AB

||는 하나라도 NULL이면 전체가 NULL이다

02 형변환과 숫자 함수

CAST - 데이터 타입 변환

CAST 함수와 :: 연산자는 같은 형변환 기능이고 ::가 PostgreSQL 고유 문법으로 더 간결하다.

```
-- CAST 함수 방식
SELECT CAST('2015-01-01' AS DATE);
SELECT CAST(42 AS VARCHAR);
SELECT CAST(3.7 AS INTEGER); -- 4 (반올림)

-- :: 연산자 방식 (PostgreSQL 고유, 더 간결)
SELECT '2015-01-01'::DATE;
SELECT 42::VARCHAR;
SELECT 3.7::INTEGER; -- 4
```

공식 문서: 타입 변환

정수로 변환하면 반올림된다

소수를 INTEGER로 변환하면 소수점이 잘려 나가는 것이 아니라 **반올림**된다.

식	결과
3.2::INTEGER	3
3.7::INTEGER	4
3.5::INTEGER	4
-3.7::INTEGER	-4
TRUNC(3.7)	3

소수점 아래를 버리려면 형변환이 아니라 TRUNC를 쓴다.

자주 쓰는 형변환 패턴

```
-- 날짜 문자열 → DATE (INTERVAL은 날짜 함수 섹션에서 다룬다)
SELECT '2015-01-01'::DATE + INTERVAL '1 month';

-- 정수 → 소수 (나눗셈에서 소수점 유지)
SELECT 1::NUMERIC / 3; -- 0.333...
SELECT 1 / 3; -- 0 (정수 나눗셈)

-- 숫자 → 문자
SELECT 42::TEXT || '번 설비';
```

정수끼리 나누면 소수점이 버려진다. 한쪽을 ::NUMERIC으로 변환하면 소수점이 유지된다.
비율 계산에서 가장 자주 만나는 실수이므로 반드시 기억한다.

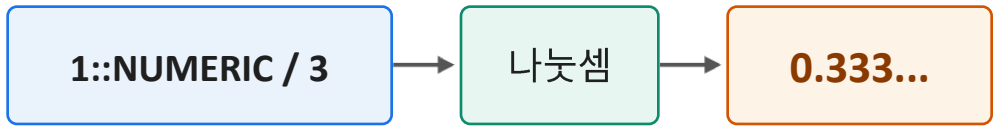
정수 나눗셈과 형변환

정수 ÷ 정수



.333... 버려짐

NUMERIC ÷ 정수



정수로 변환하면 반올림된다

- 3.2 → 3
- 3.7 → 4
- 3.5 → 4
- TRUNC(3.7) → 3

비율 계산은 한쪽을 ::NUMERIC으로 바꾼 뒤 나눈다

숫자 함수 전체 목록

함수	역할	예시	결과
ROUND(라운드)	반올림	ROUND(3.7)	4
CEIL(실)	올림	CEIL(3.2)	4
FLOOR(플로어)	내림	FLOOR(3.9)	3
TRUNC(트렁크)	소수점 버림	TRUNC(3.9)	3
ABS(앱스)	절대값	ABS(-42)	42
MOD(모드)	나머지	MOD(10, 3)	1
POWER(파워)	거듭제곱	POWER(2, 10)	1024
SQRT(스퀘어루트)	제곱근	SQRT(16)	4
GREATEST(그레이티스트)	여러 값 중 최대	GREATEST(1,5,3)	5
LEAST(리스트)	여러 값 중 최소	LEAST(1,5,3)	1

공식 문서: 수학 함수

반올림 · 올림 · 내림 · 버림

```
SELECT
ROUND(3.7) AS 반올림, -- 4
CEIL(3.2) AS 올림, -- 4
FLOOR(3.9) AS 내림, -- 3
TRUNC(3.9) AS 버림; -- 3
```

FLOOR와 TRUNC의 차이

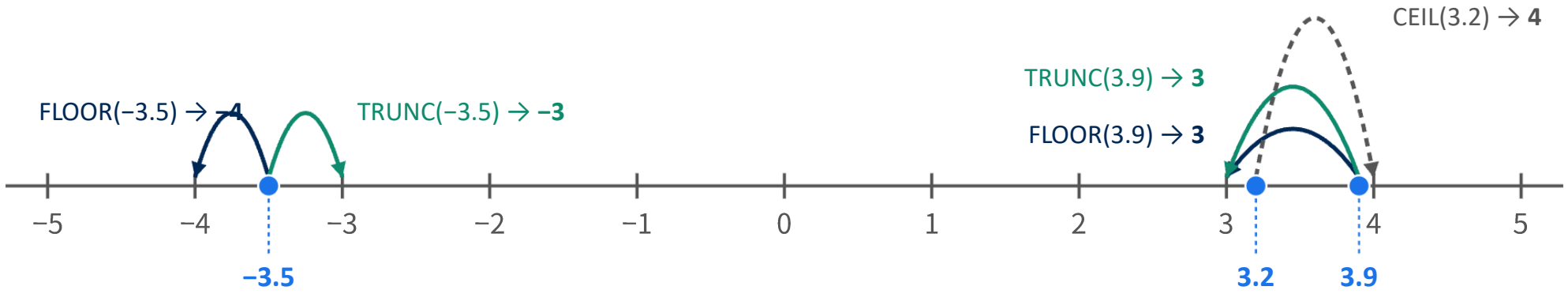
양수에서는 같지만 음수에서 결과가 다르다.

값	FLOOR	TRUNC
3.9	3	3
-3.5	-4	-3

```
SELECT
FLOOR(-3.5) AS FLOOR결과, -- -4 (더 작은 쪽으로)
TRUNC(-3.5) AS TRUNC결과; -- -3 (소수점만 제거)
```

FLOOR는 항상 더 작은 정수로 내린다. TRUNC는 소수점만 잘라낸다.

FLOOR · TRUNC · CEIL의 방향



FLOOR는 항상 더 작은 정수로, TRUNC는 소수점만 자른다

자릿수 지정 반올림

```
SELECT
ROUND(3.14159::NUMERIC, 2) AS 소수2자리, -- 3.14
ROUND(3.14159::NUMERIC, 4) AS 소수4자리, -- 3.1416
ROUND(12345.678::NUMERIC, -2) AS 백단위반올림; -- 12300
```

자릿수를 음수로 주면 정수부를 반올림한다.

GREATEST와 LEAST

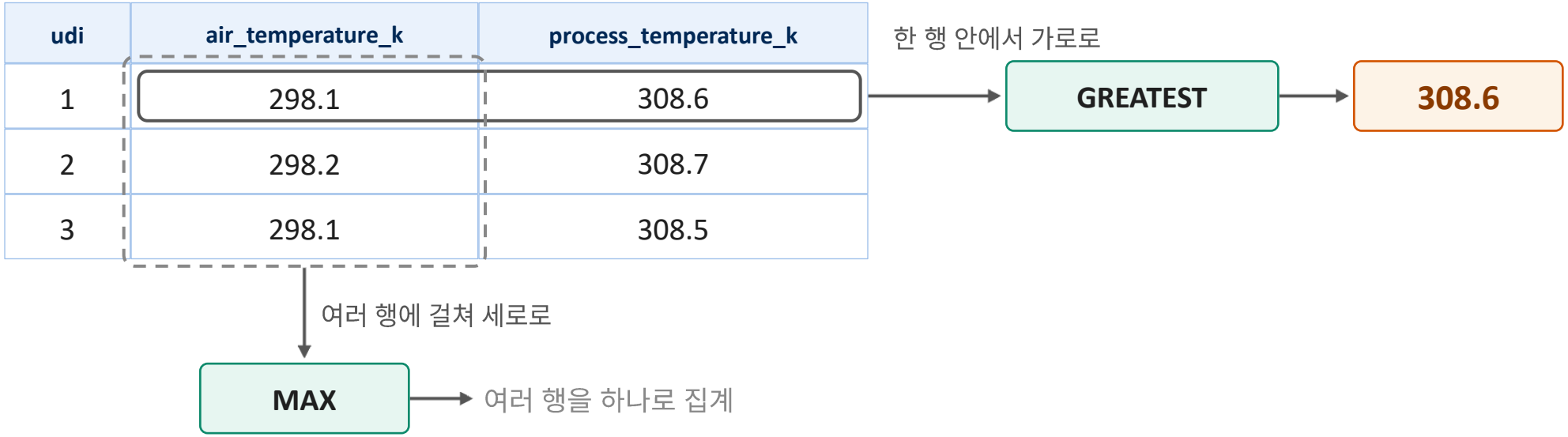
여러 컬럼 중 최대값이나 최소값을 구할 때 쓴다.

```
SELECT
GREATEST(1, 5, 3) AS 최대값, -- 5
LEAST(1, 5, 3) AS 최소값; -- 1
```

MAX와 MIN은 여러 행을 집계하지만 GREATEST와 LEAST는 한 행 안의 여러 컬럼을 비교한다.

MAX · MIN과 GREATEST · LEAST

구분	대상	예시
MAX / MIN	여러 행	MAX(unit_price)
GREATEST / LEAST	한 행의 여러 컬럼	GREATEST(a, b, c)



GREATEST는 한 행 안에서 가로로, MAX는 여러 행에 걸쳐 세로로 비교한다

기타 계산 함수

```
SELECT
ABS(-42) AS 절대값, -- 42
MOD(10, 3) AS 나머지, -- 1
POWER(2, 10) AS 거듭제곱, -- 1024
SQRT(16) AS 제곱근; -- 4
```

03 날짜 함수

현재 날짜와 시간

```
SELECT
CURRENT_DATE AS 오늘날짜, -- 2026-07-25
CURRENT_TIME AS 현재시간, -- 14:30:00
CURRENT_TIMESTAMP AS 현재일시, -- 2026-07-25 14:30:00
NOW() AS NOW결과; -- CURRENT_TIMESTAMP와 동일
```

공식 문서: 날짜/시간 함수

현재 날짜와 시간의 반환 타입

별칭	결과	반환 타입
오늘날짜	2026-08-04	DATE
현재시간	21:03:11.823231+09:00	TIME WITH TIME ZONE
현재일시	2026-08-04 21:03:11.823231+09	TIMESTAMP WITH TIME ZONE
NOW결과	2026-08-04 21:03:11.823231+09	TIMESTAMP WITH TIME ZONE

+09는 이 값이 한국 시간(세계 표준시보다 9시간 빠름)이라는 표시이고,
.823231은 초의 소수점 아래 자리(마이크로초, 1초의 백만분의 1 단위)다.

현재 날짜와 시간 정리

필요한 값	함수
날짜만	CURRENT_DATE
시간대 포함 현재 시각	NOW() (CURRENT_TIMESTAMP와 동일)
시간대 없는 현재 시각	LOCALTIMESTAMP

DATE_TRUNC - 날짜 단위 자르기

지정한 단위 아래를 모두 0으로 만든다.

단위	2025-03-15 14:30:45 결과
hour	2025-03-15 14:00:00
day	2025-03-15 00:00:00
week	2025-03-10 00:00:00 (월요일 기준)
month	2025-03-01 00:00:00
quarter	2025-01-01 00:00:00
year	2025-01-01 00:00:00

EXTRACT - 날짜 정보 추출

날짜에서 특정 값을 숫자로 꺼낸다.

추출 단위	의미	2025-03-15 결과
YEAR	연도	2025
MONTH	월	3
DAY	일	15
QUARTER	분기	1
WEEK	연중주차	11
DOW	요일 번호	6 (토요일)
DOY	연중 일자	74

요일 번호(DOW) 값

값	요일
0	일요일
1	월요일
2	화요일
3	수요일

값	요일
4	목요일
5	금요일
6	토요일

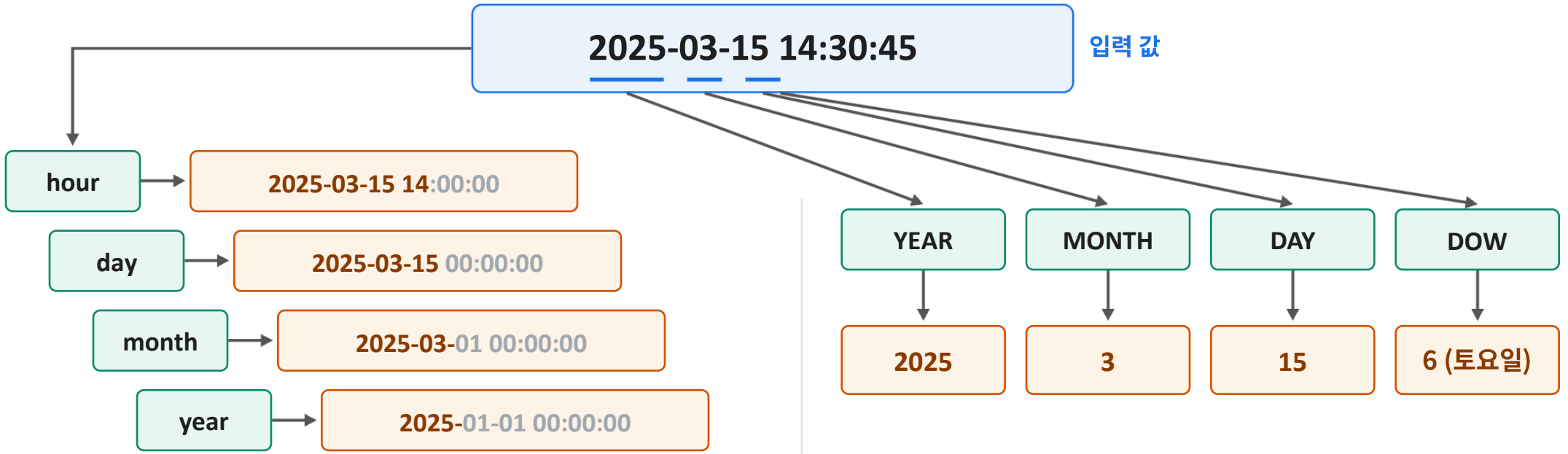
DATE_TRUNC와 EXTRACT 비교

함수	반환 타입	용도
DATE_TRUNC	timestamp	같은 달끼리 묶을 때
EXTRACT	숫자	특정 값 추출, 필터링

```
SELECT
DATE_TRUNC('month', '2025-03-15 00:00:00'::TIMESTAMP) AS DATE_TRUNC결과, -- 2025-03-01 00:00:00
EXTRACT(MONTH FROM '2025-03-15'::DATE) AS EXTRACT결과; -- 3
```

같은 달끼리 묶을 때는 DATE_TRUNC, 3월만 골라낼 때는 EXTRACT를 쓴다

DATE_TRUNC와 EXTRACT의 결과 차이



DATE_TRUNC: 아래 단위를 0으로

결과 타입 timestamp

EXTRACT: 숫자 하나를 꺼냄

결과 타입 숫자

월별 집계에는 DATE_TRUNC, 특정 달 골라내기에는 EXTRACT

INTERVAL - 날짜 계산

```
SELECT
'2025-03-15'::DATE + INTERVAL '7 days' AS 일주일후, -- 2025-03-22
'2025-03-15'::DATE - INTERVAL '1 month' AS 한달전, -- 2025-02-15
'2025-03-15'::DATE + INTERVAL '1 year' AS 일년후; -- 2026-03-15
```

사용 가능한 단위는 second, minute, hour, day, week, month, year 등이다.

AGE - 날짜 차이

```
SELECT
AGE('2015-12-31'::DATE, '2015-01-01'::DATE) AS 기간차이, -- 11 mons 30 days
'2015-12-31'::DATE - '2015-01-01'::DATE AS 일수차이; -- 364
```

AGE는 년/월/일 형태로, 빼기 연산자는 일수(정수)로 반환한다. 상황에 맞게 선택한다.

04 조건과 NULL 처리

CASE 표현식 — 개념과 기본 문법

CASE는 조건에 따라 여러 결과 중 하나를 선택해 반환하는 SQL 조건 표현식이다.

① 단순 CASE (Simple CASE)

하나의 값을 여러 후보값과 비교

```
CASE 비교할_값
    WHEN 값1 THEN 결과1
    WHEN 값2 THEN 결과2
    ELSE 기본결과
END
```

② 조건 CASE (Searched CASE)

각 WHEN의 조건식을 순서대로 검사

```
CASE
    WHEN 조건1 THEN 결과1
    WHEN 조건2 THEN 결과2
    ELSE 기본결과
END
```

읽는 순서 WHEN: 값/조건 확인 THEN: 결과 반환 ELSE: 그 밖의 경우 END: 종료

단순 CASE

```
-- 제품 품질 유형 표시
SELECT
        udi,
        product_type,
        CASE product_type
                WHEN 'H' THEN 'High'
                WHEN 'M' THEN 'Medium'
                WHEN 'L' THEN 'Low'
        ELSE '미분류'
        END AS 제품품질유형
FROM mfg.tb_ai4i
LIMIT 5;
```

udi	product_type	제품품질유형
1	M	Medium
2	L	Low
3	L	Low

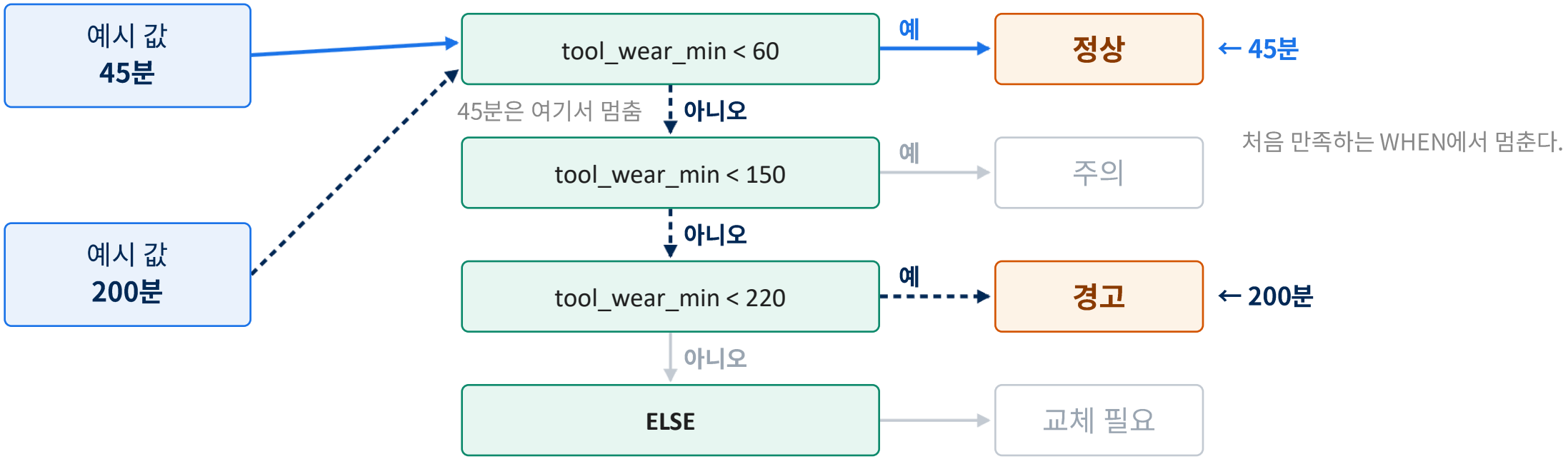
공식 문서: 조건 표현식

검색 CASE (범위 조건)

```
-- 공구 마모 단계 분류
SELECT
        udi,
        tool_wear_min,
        CASE
                WHEN tool_wear_min < 60 THEN '정상'
                WHEN tool_wear_min < 150 THEN '주의'
                WHEN tool_wear_min < 220 THEN '경고'
        ELSE '교체 필요'
        END AS 마모단계
FROM mfg.tb_ai4i
ORDER BY tool_wear_min DESC
LIMIT 10;
```

조건은 위에서부터 순서대로 평가되고 처음 만족하는 WHEN에서 멈춘다.

CASE의 위에서 아래 평가



05 형 변환

TO_CHAR

날짜·시간 또는 숫자 데이터를 지정한 서식(Format)에 따라 문자열(TEXT)로 변환하는 함수이다.

```
SELECT NOW();
-- 2026-07-25 14:30:45.123456+09 ← 보고서에 쓰기엔 지저분하다

SELECT TO_CHAR(NOW(), 'YYYY년 MM월 DD일');
-- 2026년 07월 25일 ← 읽기 좋다
```

공식 문서: 데이터 타입 형식 함수

날짜 형식 문자

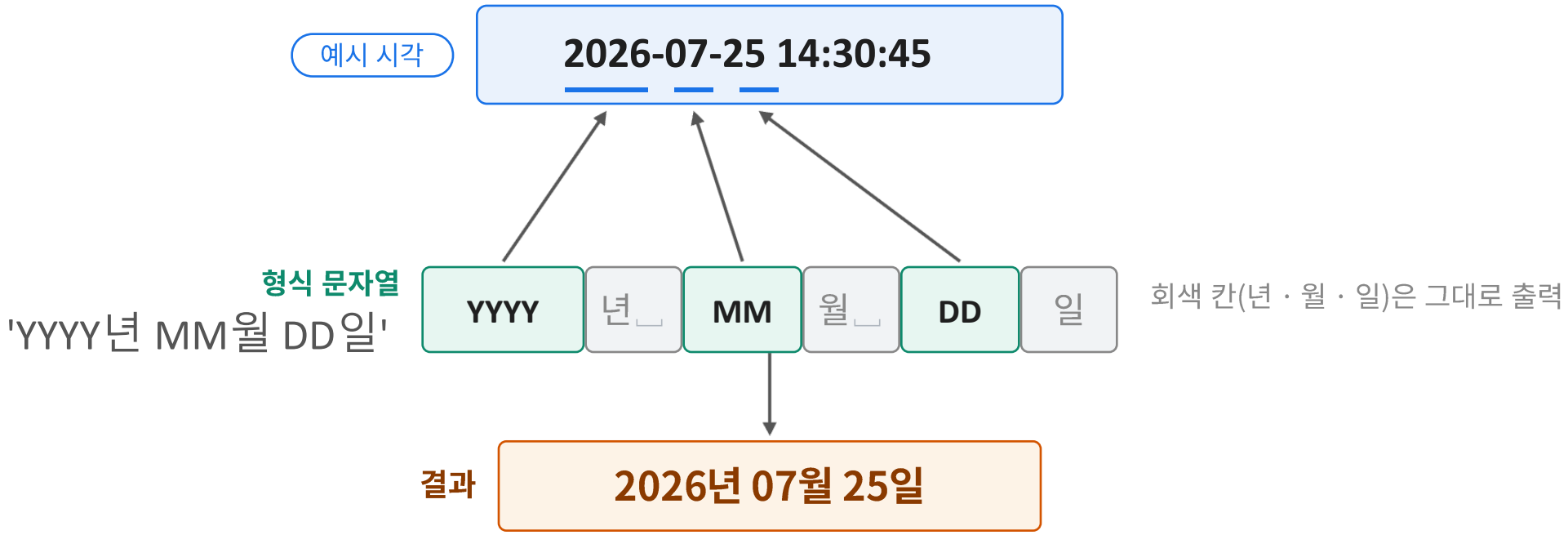
형식	의미	예시 결과
YYYY	연도 4자리	2026
YY	연도 2자리	26
MM	월 2자리	07
DD	일 2자리	25
HH24	시 (24시간제)	14
HH12	시 (12시간제)	02
MI	분	30
SS	초	45
Day	요일 전체	Saturday (9자리 고정폭)
FMDay	요일 전체 (공백 없음)	Saturday
Dy	요일 축약	Sat
Q	분기	3

Day와 Month는 가장 긴 요일·월 이름에 맞춰 자리를 채우므로 결과 뒤에 공백이 붙는다. 공백이 필요 없으면 FMDay, FMMonth처럼 앞에 FM을 붙인다.

날짜 형식 변환 예제

```
SELECT
TO_CHAR(NOW(), 'YYYY-MM-DD') AS 날짜만, -- 2026-07-25
TO_CHAR(NOW(), 'YYYY년 MM월 DD일') AS 한글형식, -- 2026년 07월 25일
TO_CHAR(NOW(), 'YYYY-MM') AS 년월, -- 2026-07
TO_CHAR(NOW(), 'HH24:MI:SS') AS 시간만, -- 14:30:45
TO_CHAR(NOW(), 'YYYY-MM-DD HH24:MI') AS 날짜시간; -- 2026-07-25 14:30
```

TO_CHAR 형식 문자 대응



DATE_TRUNC와 TO_CHAR 비교

둘 다 월 단위 표현에 쓸 수 있지만 차이가 있다.

함수	반환 타입	결과 예시	정렬
DATE_TRUNC	timestamp	2015-03-01 00:00:00	날짜 정렬 (정확)
TO_CHAR	문자열	2015-03	문자 정렬

DATE_TRUNC

다만 날짜 계산이 필요하면 DATE_TRUNC를 써야 한다.

숫자 형식 변환

```
SELECT
TO_CHAR(1234567, 'FM999,999,999') AS 천단위구분, -- 1,234,567
TO_CHAR(1234.5678, 'FM9999.99') AS 소수2자리, -- 1234.57
TO_CHAR(0.15, 'FM990.99') AS 앞자리0표시; -- 0.15
```

형식 문자	의미
9	숫자 자리 (값 없으면 공백)
0	숫자 자리 (값 없으면 0)
,	천 단위 구분자
.	소수점
FM	앞뒤 공백 제거

TO_DATE - 문자를 날짜로

```
SELECT
TO_DATE('2025-03-15', 'YYYY-MM-DD') AS 날짜변환,
TO_DATE('15/03/2025', 'DD/MM/YYYY') AS 형식지정변환;
```

CSV로 받은 날짜가 문자열로 저장된 경우 변환에 사용한다.

단순 CAST와 TO_DATE의 선택

YYYY-MM-DD처럼 PostgreSQL이 명확하게 해석할 수 있는 표준적인 날짜 문자열은 '2025-03-15'::DATE처럼 단순 형변환하는 것이 더 간단하다.
입력 형식이 15/03/2025처럼 별도 형식 규칙을 필요로 할 때 TO_DATE(문자열, 형식)을 사용한다.

공식 문서: Data Type Formatting Functions

핵심 정리 (1/2)

1) 문자열 함수: UPPER, LOWER, LENGTH, TRIM, SUBSTRING, REPLACE, CONCAT
2) 문자열 연결 ||은 NULL이 하나라도 있으면 전체가 NULL이 되고, CONCAT은 NULL을 무시한다.
3) FLOOR와 TRUNC는 음수에서 결과가 다르다.
4) GREATEST와 LEAST는 한 행 안의 여러 컬럼을 비교한다. MAX와 MIN(여러 행 집계)과 다르다.
5) 날짜 함수: CURRENT_DATE, DATE_TRUNC, EXTRACT, INTERVAL, AGE
6) EXTRACT의 DOW는 0이 일요일이다.
7) DATE에 INTERVAL을 더하면 TIMESTAMP가 된다.
8) CASE WHEN은 조건에 따라 다른 값을 반환하며 위에서부터 순서대로 평가된다.

핵심 정리 (2/2)

9) COALESCE는 첫 번째 NULL이 아닌 값을 반환한다.

10) NULLIF는 두 값이 같으면 NULL을 반환한다. 0으로 나누기 방지에 사용한다.

11) CAST와 :: 연산자는 데이터 타입을 변환한다. 정수 나눗셈에는 ::NUMERIC이 필요하다.

12) DATE_TRUNC 결과에는 시각이 붙으므로 날짜만 필요하면 ::DATE를 붙인다.

13) 소수를 INTEGER로 형변환하면 반올림된다. 버리려면 TRUNC를 쓴다.

14) TO_CHAR는 날짜와 숫자를 원하는 형식의 문자열로 바꾸고, TO_DATE는 문자열을 날짜로 바꾼다.